

CS F407 – Logical Planning Lab: Logic + Search = Planning

Name: Aditeya Kayal | **ID:** 2024A8PS0689G | **Course:** CS F407 Artificial Intelligence

This report presents the solution to the laboratory exercise [logic_lab.pdf](#).

File	Contents
logic_planner.py	STRIPS-style planner (BFS), independent plan validator, and Tasks 0, 1, 3 and 5
results.txt	Full output of <code>python logic_planner.py</code>
planner.pl , road.pl	Prolog programs for the optional extension (Tasks 6–8)
prolog_check.py , prolog_results.txt	A small backward-chaining evaluator that answers the Prolog queries, and its output
README.md / REPORT.pdf	This report, in Markdown and PDF form

Requirements: Python 3 (standard library only); SWI-Prolog (optional) for the `.pl` files. To reproduce the results:

```
python logic_planner.py
```

```
python prolog_check.py
```

Task 0 – Specification of the planning problem

- (a) Initial state:** $I = \{At(Robot, A), At(Package, A)\}$
- (b) Goal:** $G = \{At(Package, C)\}$
- (c) Actions:** `Move(A, B)`, `Move(B, A)`, `Move(B, C)`, `Move(C, B)`; `PickUp(Package, x)` and `Drop(Package, x)` for $x \in \{A, B, C\}$.
- (d) Preconditions and effects:**

Action	Preconditions	Positive effects	Negative effects
<code>Move(x, y)</code> , for connected x, y	<code>At(Robot, x)</code>	<code>At(Robot, y)</code>	<code>At(Robot, x)</code>
<code>PickUp(Package, x)</code>	<code>At(Robot, x)</code> , <code>At(Package, x)</code> , <code>¬Holding(Package)</code>	<code>Holding(Package)</code>	<code>At(Package, x)</code>
<code>Drop(Package, x)</code>	<code>At(Robot, x)</code> , <code>Holding(Package)</code>	<code>At(Package, x)</code>	<code>Holding(Package)</code>

The negative precondition `¬Holding(Package)` on `PickUp` is a small addition to the handout's specification. It prevents the robot from picking up a package it is already holding.

Applicability in the initial state (computed by the program):

Action	Applicable in I?	Unsatisfied preconditions
Move(A, B)	Yes	–
PickUp(Package, A)	Yes	–
Drop(Package, C)	No	At(Robot, C), Holding(Package)
All other actions	No	e.g. Move(B, C) requires At(Robot, B)

Answer to the question. PickUp(Package, A) is applicable, because both of its preconditions, At(Robot, A) and At(Package, A), are members of I. Drop(Package, C) is not applicable: the robot is not at C, and it is not holding the package, so $I \neq \text{Preconditions}(\text{Drop}(\text{Package}, C))$.

Think About It. An action's presence in the action list only makes it a *candidate*. It becomes applicable only when $I \models \text{Preconditions}(a)$, that is, when every positive precondition is in the state and no negative precondition is. This entailment check is the point at which logical reasoning enters the planning process.

Task 1 – Plan constructed by hand

The plan is PickUp(Package, A), Move(A, B), Move(B, C), Drop(Package, C):

State	Action taken	Facts
S0	–	At(Robot, A), At(Package, A)
S1	PickUp(Package, A)	At(Robot, A), Holding(Package)
S2	Move(A, B)	At(Robot, B), Holding(Package)
S3	Move(B, C)	At(Robot, C), Holding(Package)
S4	Drop(Package, C)	At(Robot, C), At(Package, C)

$S4 \models G$, so the plan is valid. The independent validator in the program confirmed every step.

The handout mentions the actions Move(A, B), PickUp(Package, B), Move(B, C), Drop(Package, C) as examples of what one "might need to reason about". Executed in that order, the sequence is **not** a valid plan. The validator rejects it at step 2: PickUp(Package, B) requires At(Package, B), but the package is still at A. This is exactly the kind of error that a precondition check is designed to catch.

Task 2 – LLM implementation of the planner

LLM used: Claude (Anthropic).

Prompt (as suggested in the handout):

I want to implement a simple planning agent in Python. Represent a state as a set of logical propositions. Each action should contain: a name; positive preconditions; negative preconditions; positive effects; negative effects. An action is applicable if all of its preconditions are satisfied by the current state. When an action is applied: 1. remove its negative effects from the state; 2. add its positive effects to the state. Use breadth-first search to find a sequence of actions that achieves a specified goal. The program should also detect when no plan exists; print the resulting sequence of actions; print the states reached after each action. Explain the implementation and identify any assumptions you make.

Assumptions stated with the implementation:

- the closed-world assumption (a proposition not in the state is false);
- ground (fully instantiated) actions;
- deterministic effects;
- a goal expressed as a conjunction of positive propositions, satisfied when $G \subseteq S$.

Think About It – where the specification appears in the code.

Idea	Code in <code>logic_planner.py</code>
Preconditions: when is an action applicable?	<code>Action.applicable(): pos_pre <= state and not (neg_pre & state)</code>
Effects: how does the state change?	<code>Action.apply(): (state - delete) \ add</code>
Goal: when does planning terminate?	<code>plan_bfs(): if goal <= state ($G \subseteq S$, i.e. $S \models G$), checked when a state is dequeued</code>
BFS: how are alternative plans explored?	<code>plan_bfs(): a FIFO deque of states, with a parent dictionary that records each reached state once</code>

Task 3 – Testing the generated planner

Every plan returned by the planner was re-executed by `validate()`, a function independent of the search. It checks each precondition in the state where the action is executed, and checks the goal at the end.

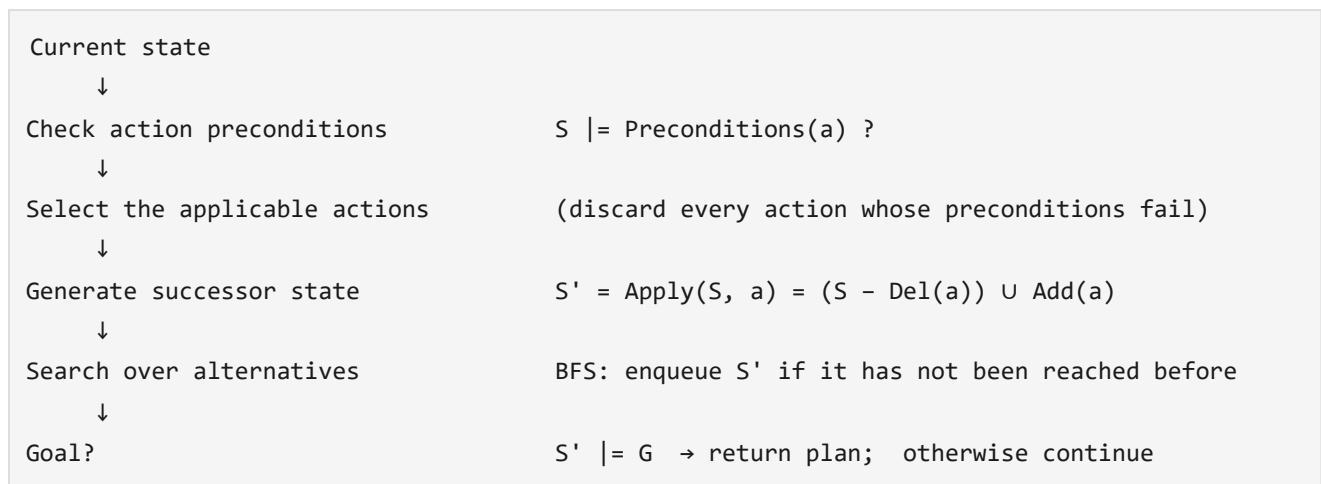
Test	Initial state	Goal	Plan found?	Resulting plan	Plan valid?
A. Original problem	At(Robot, A), At(Package, A)	At(Package, C)	Yes (4 steps; 8 states explored)	PickUp(Package, A), Move(A, B), Move(B, C), Drop(Package, C)	Yes
B. PickUp removed	same	At(Package, C)	No plan found (3 states explored)	–	–
C1. Irrelevant actions added: Move(A, C) (robot only) and Wait	same	At(Package, C)	Yes (3 steps)	PickUp(Package, A), Move(A, C), Drop(Package, C)	Yes
C2. Irrelevant actions added, PickUp removed	same	At(Package, C)	No plan found	–	–
C3. Control: different goal	same	At(Robot, C)	Yes (2 steps)	Move(A, B), Move(B, C)	Yes

Discussion.

- **Test A** returned the same plan that was constructed by hand. Every step was verified.
- **Test B.** Without PickUp the package can never leave A. The planner explored the three reachable states (robot at A, B, or C, with the package at A) and correctly reported that no plan exists, rather than inventing an action.
- **Test C.** The added actions move the robot without moving the package.
- In C1 the planner used the robot-only shortcut, but it still inserted PickUp before the move and Drop after it. Robot movement alone was therefore not treated as package movement.
- Test C2 is the decisive case. The robot *can* reach C, but the package cannot, and the planner reports failure.
- Test C3 shows the contrast: when the goal really is At(Robot, C), two moves suffice.
- Together these tests confirm that the planner does not treat At(Robot, C) as equivalent to At(Package, C).

Task 4 – Logic and search

The completed description is:



The missing step is **filtering to the applicable actions**: only actions whose preconditions are entailed by the current state are used to generate successors.

How logic and search cooperate.

- **Logic** decides what is *possible*. Given a state, the entailment check $S \models \text{Preconditions}(a)$ determines which actions may be executed. The effect rules then determine exactly what the world looks like afterwards, and the goal test $S \models G$ determines when the task is complete.
- **Search** decides what to *try*. Among the many sequences of applicable actions, BFS explores them systematically in order of length. It records visited states so that no state is explored twice, and it returns the first (and therefore shortest) sequence that reaches a goal state.

Logic alone cannot choose a sequence of actions, and search alone would generate physically impossible sequences. Planning requires both, which is the same state-space search as in the previous module with the transition function defined logically.

Task 5 (optional) – Can the LLM verify its own plan?

The LLM was asked: *"For every action in the plan, identify its preconditions and show that those preconditions are satisfied in the state in which the action is executed."* Its explanation of the four-step plan was:

1. PickUp(Package, A) requires At(Robot, A) and At(Package, A); both hold in S_0 .
2. Move(A, B) requires At(Robot, A), which still holds in S_1 because PickUp does not move the robot.
3. Move(B, C) requires At(Robot, B), made true by the previous move.
4. Drop(Package, C) requires At(Robot, C) and Holding(Package); the former was made true by Move(B, C), and the latter by PickUp and never deleted.

This explanation agrees with the state transitions $S_0 \rightarrow S_4$ computed by the program (Task 1 table).

However, an explanation and a verification are not equivalent. To illustrate this, the validator was given a plausible but incorrect plan: PickUp(Package, A), Move(A, C), Drop(Package, C). A natural-language justification of this plan reads convincingly ("the robot picks up the package, moves to C, and drops it").

The validator rejects it immediately, because `Move(A, C)` does not exist: A and C are not connected. It rejected the handout's example sequence in the same way (Task 1).

Which should be trusted more? The **independently executed state transitions**. The program applies the formal definitions of applicability and effect mechanically to every step, and its result depends only on the specification. An LLM's explanation is generated text. It can be fluent and still overlook an unsatisfied precondition or refer to an action that is not in the domain, and it cannot be relied upon to check itself. As the handout states, *a generated explanation is not the same as an independent verification*.

Reflection questions

1. Why specify preconditions and effects before asking an LLM to write the planner? The preconditions and effects *are* the problem specification. With them written down, the generated code can be checked against a precise reference. Without them, the LLM must guess the domain rules, and an incorrect guess is hard to detect because the code may still run and produce plausible output. For example, it might omit `¬At(Package, x)` from `PickUp`, or allow `Drop` without `Holding`.

2. An error that could occur if preconditions were not checked. The planner could return `Drop(Package, C)` as a one-step plan from the initial state. The robot is at A and is not holding the package, yet the effect `At(Package, C)` would be added and the goal would appear to be satisfied. Similarly, `Move(A, C)` could be accepted even though A and C are not connected.

3. Why is a plan that "looks reasonable" not necessarily valid? Validity is a precise property: every action must be applicable in the state produced by the preceding actions, and the final state must satisfy the goal. A sequence can contain all the right actions in the wrong order, as the handout's example `Move(A, B), PickUp(Package, B), ...` does, or use an action that looks sensible but does not exist in the domain, such as `Move(A, C)`. Both look reasonable, and both fail formal validation.

4. What did the LLM contribute to the implementation? It produced the state and action data structures, the applicability and effect functions, the BFS loop with parent tracking, plan extraction, and the output formatting. It also stated its modelling assumptions, such as the closed-world assumption and ground actions.

5. What had to be verified independently?

- that applicability is decided exactly by the specified preconditions, including the negative ones;
- that effects delete before they add;
- that the planner reports failure when no plan exists (Test B);
- that it does not confuse the robot's location with the package's (Tests C1–C3);
- that every returned plan is valid when re-executed step by step by a separate validator.

6. Where is logical reasoning used in this laboratory?

- In the entailment checks $S \models \text{Preconditions}(a)$ (action applicability) and $S \models G$ (goal test).
- In the effect rules that determine the successor state.
- In the Prolog extension, where facts and rules are used to infer whether a proposed move is supported by the warehouse knowledge.

7. How is planning related to the search algorithms of the previous module? Planning is state-space search in which the states are sets of propositions and the transition function is defined by action

preconditions and effects. The same BFS algorithm, frontier, visited set, and path reconstruction are used, and the same trade-offs apply. BFS finds shortest plans but its cost grows exponentially with plan length, so larger domains need heuristics (as with A*).

Optional extension – Prolog as a logical verifier

SWI-Prolog was not installed on the machine used for this report. The queries were therefore evaluated with `prolog_check.py`, a small backward-chaining (SLD-resolution) evaluator that reads the same `planner.pl` and `road.pl` files; its output is in `prolog_results.txt`. The outcomes are identical to those that SWI-Prolog gives for these programs, and they can be confirmed by loading the files with `swipl planner.pl`.

Task 6 – Prolog facts and the `can_move` rule

Query	Result
<code>?- can_move(a,b).</code>	true
<code>?- can_move(a,c).</code>	false

(a) `can_move(a,b)` is true because it unifies with the head of the rule `can_move(X,Y) :- connected(X,Y)` under $X = a, Y = b$. The resulting subgoal `connected(a,b)` matches a fact in the knowledge base.

(b) `can_move(a,c)` cannot be established because the only way to prove it is through the subgoal `connected(a,c)`, which is neither a fact nor derivable. Prolog uses the closed-world assumption and *negation as failure*: what cannot be proved is reported as false. Note that the program contains no transitivity rule, so a path through B does not make `can_move(a,c)` true.

(c) The rule is the Horn clause $\forall X \forall Y (\text{Connected}(X, Y) \rightarrow \text{CanMove}(X, Y))$. In Prolog the conclusion is written first, so head `:- body` corresponds to $body \rightarrow head$.

Task 7 – Using Prolog to check a proposed plan

Query	Result
<code>?- valid_move(a,b).</code>	true
<code>?- valid_move(b,c).</code>	true
<code>?- valid_move(a,c).</code>	false

Challenge. The proposed action `Move(a, c)` is **not supported** by the warehouse knowledge, because `valid_move(a,c)` fails: there is no fact `connected(a,c)`. The same conclusion was reached by the Python validator in Task 5. The file `planner.pl` also contains a recursive predicate `valid_plan/1`, which checks an entire route (a list of locations) in one query.

Think About It. In this architecture the Python planner (and, indirectly, the LLM that helped write it) *generates* a candidate, and Prolog *checks* it against a separately written logical description of the

world. The value of the verifier lies in its independence: an error in the generator is caught because the checker relies on its own encoding of the domain.

Task 8 – Connecting Prolog to logical reasoning

The query `?- reduce_speed.` **succeeds**. Prolog resolves `reduce_speed` using the rule `reduce_speed :- slippery`, which produces the subgoal `slippery`. That subgoal is resolved using `slippery :- wet_road`, which produces `wet_road`, and `wet_road` is a fact. All subgoals are therefore proved.

As a chain of implications:

wet_road (fact) $\Rightarrow$ [wet_road $\rightarrow$ slippery] $\Rightarrow$ [slippery $\rightarrow$ reduce_speed] $\Rightarrow$ reduce_speed (conclusion)

This is two applications of *modus ponens*.

Think About It. In this laboratory the programmer supplied only facts and rules, and the Prolog engine performed the inference. As the handout notes, Prolog is not identical to classical logic: its depth-first search, clause ordering, and negation as failure mean that some programs behave differently from classical entailment. For example, a left-recursive rule can loop indefinitely. For the ground Horn clauses used here, however, the answers coincide with classical logical consequence.

Reflection on the extension

1. What is the difference between a Prolog fact and a Prolog rule? A fact, such as `connected(a,b).`, states unconditionally that something is true. A rule, such as `can_move(X,Y) :- connected(X,Y).`, states that the head is true *if* the body is true, which is a conditional (implication).

2. How does a Prolog query correspond to asking whether something follows from a knowledge base? A query `?- q.` asks whether $KB \models q$. Prolog attempts to prove q by resolution from the facts and rules, answering *true* if a proof is found (and returning variable bindings if there are any) and *false* otherwise.

3. Why use a Prolog program to verify a plan generated by a Python program? The verifier encodes the domain declaratively and independently of the planner's implementation. A bug in the Python successor function would not be reproduced in the Prolog knowledge base, so a disagreement between the two exposes it. Declarative rules are also short and easy to inspect.

4. What advantage does an independent verifier provide when the plan was generated with the help of an LLM? LLM-generated code and LLM explanations can be plausible but wrong, and the LLM cannot be relied upon to detect its own errors. An independent, formal verifier provides a check whose correctness does not depend on the generator. This is the *generate $\rightarrow$ independently verify* pattern: the LLM contributes speed, and the verifier contributes trust.